Lappeenrannan teknillinen yliopisto
School of Business and Management

Sofware Development Skills

Andrei Ziber, 2209518

LEARNING DIARY, Software Development Skills: Mobile 2025-26 MODULE

LEARNING DIARY

Day 1
I started learning Android Studio, an IDE that includes a text editor, debugging tools, and an emulator. I set up my first project to accept two numbers and display their sum. I learned how the project structure organizes files into Java code, resources (such as the res folder for layouts and strings), and the AndroidManifest.xml which specifies the starter launcher activity. I built the user interface using a ConstraintLayout to align EditText, Button, and TextView components. In the Java code, I connected these UI elements using the findViewById method and processed user interactions by setting a setOnClickListener. I also practiced debugging an app crash. I learned how to set breakpoints inside the onClick method and step through code using "step over", "step into", and "step out" commands to inspect variable states and fix an error where an integer was passed instead of a string.

Day 2
I learned about two core Android elements: Activities, which act as display screens, and Intents, which serve as requests to perform an action. I built a "Quick App Launcher" to practice navigating between screens. I used an intent to launch a second activity and passed extra data as key-value pairs using the putExtra method. On the second screen, I retrieved this extra data to dynamically change the text to "HELLO WORLD". I also learned how to launch external applications outside of my app. I created a button that uses an intent with an action view, parses a URL string into a URI, and successfully opens the device's web browser.

Day 3
I learned how to create interactive lists using the ListView component. I stored item names, prices, and descriptions as string arrays within the strings.xml resource file. To display these details, I created a custom row layout using a RelativeLayout and built a custom class named ItemAdapter that extends BaseAdapter. Inside this adapter, I used a LayoutInflater to inflate the layout and inject the corresponding strings into the text views. I made the list interactive by adding a setOnItemClickListener so that tapping an item passes its index via an intent to open a detail activity with a picture. Finally, I learned how to safely handle large, high-resolution images that could otherwise cause the app to crash due to memory overload. I used BitmapFactory.Options with inJustDecodeBounds to read the image dimensions without loading it fully into memory, calculated a scaling ratio against the screen width, and then safely decoded the scaled image.